

1 Normalizing Flows

An important goal in statistical modeling is sampling, where we generate random samples from an probability distribution p_x^* of interest. Very often the distribution is not tractable and we have to rely on Monte Carlo methods like the Metropolis-Hastings algorithm, where knowing a function proportional to p_x^* is sufficient. On the side of machine learning there exist a variety of different approaches to the task of sampling. These models are usually referred to as generative models. In general they all try to learn the target probability distribution p_x^* and then sample from it. One of the earliest examples are Boltzmann machine, in which we minimize an energy function, which resembles the one of the Ising model. The probability of the visible units in the restricted Boltzmann machine will then follow a Boltzmann distribution. They are however not as general as other approaches like GANs, because one has to make sure the units match the physical system of interest. GANs on the other hand can be used quite flexibly for different datasets. These models however have one major problem. We do not have direct access to the probability distribution. A more recent approach are the so-called normalizing flows [12]. These neural networks are trying to directly learn the distribution function and are tractable because the networks are invertible.

1.1 Invertible networks

A very detailed overview of these types of networks is [10], which we will follow here. We will use the notation that is most common in the machine learning literature, namely x for the random variables of interest and z for the latent variables. We follow the generative direction, where we start with z and get x as an output. The z variables are drawn from a base (or prior) distribution, which is usually taken to be a Gaussian $\mathcal{N}(0, 1)$

$$p_z(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2} . \tag{1.1}$$

We will then apply a differentiable and invertible transformation G on z , which maps them to the output $x = G(z)$. The probability distribution of the output can be calculated with the help of the Jacobian by simply applying the chain rule

$$\begin{aligned} p_x(x) &= p_z(z) \left| \det \frac{\partial G}{\partial z} \right|^{-1} = p_z(G^{-1}(x)) \left| \det \frac{\partial G^{-1}}{\partial x} \right| \\ &= p_z(z) |\det J_G(z)|^{-1} = p_z(G^{-1}(x)) |\det J_{G^{-1}}(x)| , \end{aligned} \quad (1.2)$$

where the last equality is valid because of the inverse function theorem and J_G is the matrix of all partial derivatives. The extension to multiple bijective maps G_i is straightforward and is done by simple composition of transformations. The full transformation of the network consisting of K invertible layers is then given by

$$G = G_K \circ \dots \circ G_1 \quad (1.3)$$

with inverse transformation

$$G^{-1} = G_1^{-1} \circ \dots \circ G_K^{-1} . \quad (1.4)$$

The determinant of the Jacobian $J_G = \frac{\partial G}{\partial z}$ is still easily accessible and is a product of each layer

$$\det J_G(z) = \det J_{G_K}(G_{K-1}(z_{K-2})) \dots \det J_{G_1}(z) , \quad (1.5)$$

where we defined $z_i = G_i(z_{i-1})$ and identified $z_0 \equiv z$, $z_K \equiv x$. The normal distribution is therefore transformed into a different distribution p_x . Such a mapping exists, if p_z and p_x are well-behaved [10]. Then the prior can even be taken to be the uniform distribution. The design of the bijective mappings will then be guided by the idea, that one can compute its inverse and Jacobians easily.

1.2 Unitary flows

When choosing a transformation for the layers G_k , one usually opts for simple maps with tractable Jacobian determinant. They should also be composable, so we can easily extend the network in depth. As only the logarithm of the Jacobian determinant appears in the KL divergence, we track the logarithm instead of the Jacobian for each layer. The direct computation of $\log |\det J|$ is more stable and prevents exploding arguments, since the determinant can easily blow up in magnitude. Applying the logarithm to (1.5) gives us

$$\log |\det J_G(z_0)| = \sum_K \log |\det J_{G_k}(z_{k-1})| . \quad (1.6)$$

We can easily collect the log determinants in the generation pass for each layer and sum up all the terms.

One possible choice for G_k are linear maps. Other possibilities are RNVPs or autoregressive flows for example. Linear maps can have additional constraint on them to make the computation of the inverse or determinant easier. We will make use of orthogonal or unitary matrices as for them $|\det J| = 1$ holds. Assuming we have a upper triangular matrix with zeros on the diagonal

$$\tilde{M} = \begin{pmatrix} 0 & m_{1,2} & \dots & m_{1,n} \\ 0 & 0 & \ddots & \vdots \\ \vdots & & \ddots & m_{n-1,n} \\ 0 & 0 & \dots & 0 \end{pmatrix} \quad (1.7)$$

This matrix can be made skew-hermitian (or anti-hermitian) by

$$M = \tilde{M} - \tilde{M}^\dagger . \quad (1.8)$$

The space of skew-hermitian matrices with zero trace form the Lie algebra of the special unitary group $SU(n)$. So we can apply the exponential map on M and get the unitary layer for the flow:

$$U = e^M . \quad (1.9)$$

Another possibility to construct a unitary matrix from M is to use the so-called Cayley transform, which is defined as

$$U = (I - M)^{-1}(I + M) , \quad (1.10)$$

where I is the identity matrix. The inverse transformation can be found easily and it reads

$$M = (U - I)(U + I)^{-1} . \quad (1.11)$$

One should note, that we cannot generate any element of $SU(n)$ by this transformation, since we are excluding certain matrices, for example those which are unitary but not skew-hermitian. Both options, the Cayley and exponential map, take $\mathcal{O}(n^3)$ time to compute, which will become inefficient for large matrices.

1.3 Kullback-Leibler divergence

In order to use and train a flow network for an inference or modeling task, an appropriate choice for the loss function has to be made. The most common function, which is used, is the relative entropy or Kullback-Leibler divergence. It serves as a measure for how much two distributions differ from each other. It is important to know that the divergence is not symmetric in its arguments. For flow networks we can distinguish between the forward KL divergence and the reversed one. The forward version is defined as

$$\begin{aligned} D_{\text{KL}}(p_x^*(x) \parallel p_x(x)) &= \sum_x p_x^*(x) \log \left(\frac{p_x^*(x)}{p_x(x)} \right) \\ &= \mathbb{E}_{p_x^*} [\log p_x^* - \log p_x] \\ &= \mathbb{E}_{p_x^*} [\log p_x^* - \log |\det J_{G^{-1}}| - \log p_z] , \end{aligned} \quad (1.12)$$

where the expectation value over the target distribution is carried out as an averaging over the training set. The divergence is zero if the two distributions are equal. It can be interpreted as the additional entropy, if one uses the distribution p_x instead of the target p_x^* . The expectation over p_x^* can be estimated via Monte Carlo and gives us a constant term:

$$D_{\text{KL}}(p_x^*(x) \parallel p_x(x)) \approx -\frac{1}{N} \sum_{n=1}^N \log p_z + \log |\det J_{G^{-1}}| + \text{const.} \quad (1.13)$$

It is worth noting that in this approach we do not need to calculate $G(z)$, although it is of course required for sampling. In the our holographic network, which will be presented in the next chapter, we do not have a training set at hand nor do we have access to a tractable target distribution.

However we can make use of the reversed KL divergence, which includes an expectation value over the network distribution, i.e. the averaging over a sampled dataset. So for our purposes the reversed KL divergence is more suitable:

$$\begin{aligned} D_{\text{KL}}(p_x(x) \parallel p_x^*(x)) &= \mathbb{E}_{p_x} [\log p_x - \log p_x^*] \\ &= \mathbb{E}_{p_z} [\log p_z - \log |\det J_G| - \log p_x^*] . \end{aligned} \quad (1.14)$$

We notice that the estimate of the sum over p_x^* does not give a constant term anymore, since it is not weighted with p_z . But for the evaluation of the expectation value a function $\tilde{p}_x \sim p_x^*$, which is proportional to the target distribution, is sufficient, because the normalization constant will give a constant term due to the logarithm. Our divergence

then becomes

$$D_{\text{KL}}(p_{\mathbf{x}}(x) \parallel p_{\mathbf{x}}^*(x)) = \mathbb{E}_{p_z} [\log p_z - \log |\det J_G| - \log \tilde{p}_{\mathbf{x}}] + \text{const.} \quad (1.15)$$

When our model finishes training, we will end up with a network distribution p_x , which captures the information in p_x^* but is tractable. This is called probability density distillation. This will result in much faster sampling than traditional MC methods.

Since the flow network is invertible we can define another distribution p_z^* as the one induced by the network transformation: $p_x^* \xrightarrow{G^{-1}} p_z^*$. Changing variables from x to z for p_x^* in the reverse KL divergence (1.14) is simple, because the Jacobian is already provided by $\det J_G$. We then end up with the forward divergence of the latent distributions:

$$\begin{aligned} D_{\text{KL}}(p_{\mathbf{x}}(x) \parallel p_{\mathbf{x}}^*(x)) &= \mathbb{E}_{p_z} [\log p_z(z) - \log |\det J_G(z)| - \log p_{\mathbf{x}}^*(G(z))] \\ &= \mathbb{E}_{p_z} [\log p_z(z) - \log p_z^*(z)] \\ &= D_{\text{KL}}(p_z(z) \parallel p_z^*(z)) . \end{aligned} \quad (1.16)$$

That means fitting the network distribution p_x to the target p_x^* under the reversed KL divergence is equivalent to fitting the induced p_z^* to the normal distributions p_z , i.e. they are normalized under the flow.

1.4 Reparametrization trick

The latent variable $\mathbf{z}$ are in general drawn from a multivariate Gaussian

$$p_{\mathbf{z}}(\mathbf{z}) = \frac{1}{\sqrt{\det(2\pi\Sigma)}} \exp \left[-\frac{1}{2} \mathbf{z}^\dagger \Sigma^{-1} \mathbf{z} \right] , \quad (1.17)$$

where Σ is called the covariance matrix. In case that all variables are independently sampled from $\mathcal{N}(0, 1)$ the covariance will be a diagonal with the variances σ_i^2 as its entries. The evaluation of the loss function or KL divergence involve the expectation value over the prior distribution. This makes it difficult to train Σ , if one wants it to capture meaningful information. Luckily there's a simple solution, called the reparametrization trick. By changing variables with $\mathbf{z} = L\boldsymbol{\epsilon}$ we get i.i.d. variables which follow the multivariate Gaussian with unit covariance

$$p_{\boldsymbol{\epsilon}}(\boldsymbol{\epsilon}) = \frac{1}{(2\pi)^{\frac{N}{2}}} \exp \left[-\frac{1}{2} \boldsymbol{\epsilon}^\dagger \boldsymbol{\epsilon} \right] , \quad (1.18)$$

where $\Sigma = LL^\dagger$ and $\epsilon_i \sim \mathcal{N}(0, 1)$. The matrix L can be found by the so-called Cholesky decomposition, which factorizes Σ into a product of a lower triangular matrix L and an upper triangular matrix $L^\dagger$.

The multivariate Gaussian now has effectively become a linear layer with weight L and the sampling distribution has no parameters. This enables us to perform standard back-propagation to train and learn L and Σ . However, one has to be careful, when designing Σ , since it has to stay positive-definite as required of a covariance matrix.

2 Holographic Neural Network

When modeling a boundary theory and its holographic bulk with a neural network, the first step is to decide which physical quantity will be represented by the network nodes. One possibility is use the Hamilton formulation of the bulk theory and identify the units of a sequential net with the values of the bulk field and its conjugate momentum [4]. The forward direction of the network will then represent the radial coordinate z of the bulk space and the weight matrix has to be constraint, such that it produces the discrete Hamilton equations of motion. To train the model a set of observables of the boundary QFT has to be mapped to the values of the network on the input layer at $z = 0$. This has been demonstrated for a conformal QCD theory [5]. The disadvantage of this approach is that the neural network is defined on the phase space of the fields and will therefore not directly model a holographic space. The geometric information is contained in the derivatives, which are encoded in the weight constraints. The phase space representation also heavily restricts the size of the sequential net, i.e. the input dimension is fixed to two for one scalar bulk field. Moreover, sampling will be quite difficult or inefficient, as we have no access to the probability distribution of the CFT.

A better option comes from exploiting the fact, that the visible units of a trained restricted Boltzmann machine (RBM) obey the distribution $p = \frac{1}{Z}e^{-E}$, which can be used to model a QFT, when the energy function is identified with the classical limit of the gravity action [3]. The connections between the layers of the RBM have to be carefully defined by choosing a consistent discretization scheme for the field derivatives, that appear in the action. Further constraints on the network units are also needed to ensure the existence of a saddle approximation for the gravity theory. The RBM will then capture the geometry of the bulk space, since the metric components appear in the weights. The network size can now be easily scaled, because it is defined in discrete real space. While this generative model gives us the ability of sampling, the probability distribution is unfortunately not tractable.

To circumvent this problem we will use a normalizing flow, which is much more efficient and versatile because of its invertibility and tractable probability distribution. But other

than before we will not model the bulk and then infer CFT observables from the boundary values, but we will try to map the CFT degrees of freedom directly to a bulk field. To achieve that, the network will be set up as a renormalization procedure for the boundary QFT. Since we will learn the probability distribution via the use of the reversed KL divergence, there is no need to prepare a training data set. The hyperbolic geometry will be an emergent feature, that will appear after successful training.

2.1 Renormalization and discrete hyperbolic space

Tensor network MERA

Because the radial coordinate z can be interpreted as the energy scale of an RG-flow, a discrete hyperbolic space can also be at the same time a discrete renormalization. It has been found that some tensor networks have an inherent hyperbolic geometry, like the multi-scale entanglement renormalization ansatz (MERA) [13], which was first introduced as a method to find a representation of a wave function for the ground state of a critical quantum system. The idea is that a wave function $|\Psi\rangle$, defined on a lattice, will be coarse-grained, as it goes from one layer to the another. The tensor network maps $|\Psi\rangle$ to a product of nonentangled ground states $|0\rangle_i$, see ???. The wave-function will correspond to the boundary state of the holographic mapping, while the $|0\rangle_i$ will be interpreted as bulk states. Each layer consists of two type of maps: the first tensor type, called disentangler, transforms the wave-function into a less entangled state, while the second type performs the coarse-graining. For a 2D theory, they can be positioned in a overlapping manner, see ??.

The hyperbolic geometry can be deduced from the tree-like structure of the network. Let us look at the geodesic distance between two sites on the boundary, i.e. the shortest path connecting them in the bulk. We see, that the geodesic has to decay logarithmically with the boundary distance (??). Suppose we have n legs attached each bulk node, then the distance is given by

$$\mathcal{D}_{\text{MERA}}(x, y) \sim \log_n |x - y| , \quad (2.1)$$

which is consistent with a hyperbolic space. To check the ability of the network to model a CFT, we analyze the two-point function. Let's denote a tensor on the boundary site position x by ψ_x . The correlation $\mathcal{C}(\psi_x, \psi_y^*) = \langle \psi_x \psi_y^* \rangle - \langle \psi_x \rangle \langle \psi_y^* \rangle$ between two sites at x and y in general decays exponentially in terms of the geodesic distance [2]

$$\mathcal{C}(\psi_x, \psi_y^*) \sim e^{-\alpha \mathcal{D}_{\text{MERA}}(x, y)} , \quad (2.2)$$

into which we can plug in the expression from before. This leads to a polynomial decay of the correlation as one would expect of a system in critical phase:

$$\mathcal{C}(\psi_x, \psi_y^*) \sim e^{-q \log_n |x-y|} \sim \frac{1}{|x-y|^q} . \quad (2.3)$$

As it turns out, the tensor network also satisfies the area law for entropy. It seems, that MERA is a good candidate for a discrete hyperbolic space in a holographic setup. In order for this AdS/MERA correspondence to be consistent, one should identify the lattice spacing with the AdS radius L by matching geodesic lengths [1]. This means, that the network can only capture physics on scales larger than L and should be regarded as a classical spacetime.

Exact holographic mapping

Using a network of unitary maps, which are arranged in a MERA-like structure, one can define a mapping of QFT operators to operators in the bulk. This so-called exact holographic mapping (EHM) [11] also performs an RG transformation at each layer. However, in its original formulation it does not implement two types of maps like MERA. In particular the disentanglers have been omitted. Effectively, the transformations change the Hilbert space basis for the operators (and also the states). Assuming n incoming operators, the unitary map M separates the degrees of freedoms into UV and IR modes, see ???. When going from the boundary into the bulk interior, the IR degrees of freedom are fed into the next layer of transformation, while the UV modes are interpreted as bulk operators. This corresponds to a UV-cutoff of the CFT. It has been shown, that one can actually find such a map, for which the Hamiltonian will split into three terms. One of them is simply a rescaling of the original Hamiltonian and can be identified as the IR mode, since it is the result of a renormalization. The other two terms are then the UV mode and an interaction term.

Following [11] and [8], consider fermionic system on a lattice with a Hamiltonian of the form

$$H = \sum_k c_k^\dagger \hat{h}_k c_k , \quad (2.4)$$

where c_k are annihilation operators and $\hat{h}_k$ is some interaction, consisting of Pauli matrices. A coarse-graining can be easily achieved by grouping together two neighboring spinors at sites $2i$ and $2i-1$ and defining the new degree of freedom to be proportional to the sum.

A rotation like

$$\begin{pmatrix} r_i \\ b_i \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} c_{2i-1} \\ c_{2i} \end{pmatrix} \quad (2.5)$$

can map c_i into renormalized IR modes r_i and bulk UV modes b_i . If we rewrite the Hamiltonian in terms of r_i and b_i , the term, which depends only on r_i , will be of the same form as the original Hamiltonian except being rescaled with half the number of operators:

$$H_{\text{IR}} = \frac{1}{2} \sum_k r_k^\dagger \hat{h}_k r_k . \quad (2.6)$$

Thus the boundary theory has been renormalized. We can then perform the transformation (??) iteratively at each layer of the network on the IR modes r_i coming from the previous layer. With each RG step the number of degrees of freedom is reduced by half.

2.2 Holographic RG-flow as a neural network

We will implement a normalizing flow with MERA-like structure, which can perform a transformation G^{-1} on the input data similar to an EHM. The random variables (input) of our network will be field values of a boundary QFT and the latent variables will be the values of a bulk gravity theory. Our aim is to use the flow to learn an approximation for the probability density of the QFT, defined on a square lattice with N sites in total, given by

$$p_{\text{QFT}}(\psi) = \frac{1}{Z} e^{-S_{\text{QFT}}[\psi]} , \quad (2.7)$$

where S_{QFT} is the action Z the partition function. When a QFT configuration is fed to the flow, the field values ψ_i will be separated into IR-modes ψ'_i and UV-modes ϕ_i . Like in the EHM, the high energy modes ϕ_i will be interpreted as a degree of freedom of the bulk system. So the QFT undergoes a renormalization step at each layer and the flow network will model an RG-flow, see ??. The isometries of the MERA will be called *decimators*, as they actually decimate the degree of freedoms of the field theory. The latent variables ϕ_i will be sampled from a multivariate Gaussian, which will ultimately contain the bulk information. The training will be done as a probability density distillation by using the reversed direction of the net. We will test various modifications of the flow, such as different choices for disentanglers and decimators or designs of the prior distribution. Because we have translation invariance within an RG-layer, we will choose all disentangler or decimators within one layer to be the same. Basically they function like filters, which run across the CFT configuration like a CNN.

We will maintain a physics motivated notation, so we have the following identifications with the notation commonly used in machine learning: the random variable $x \equiv \psi$, the latent variable $z \equiv \xi$, the target distribution $p_x^*(x) \equiv p_{\text{QFT}}(\psi)$, the base distribution $p_z(z) \equiv p_{\text{base}}(\xi)$ and the density, that the network has learnt $p_x(x) \equiv p_{\text{flow}}(\psi)$. The filters of disentanglers/decimators will be denoted by D .

The training will happen in two stages. First in stage I we will take the bulk distribution to be i.i.d. normal distributions for each variable and train the network transformation G . This is the standard approach in probability density distillation with normalizing flows. In stage II the network weights will be fixed and we will try to capture the correlation of the bulk variables by training the covariance matrix via the reparametrization trick. The off-diagonal components of the covariance Σ will then reveal the hyperbolic geometry by calculating geodesic distances in the bulk from them.

Such a neural network design was first implemented in [9], where it was applied to spin states. It was later generalized [7] to other data sets, which are more common in the machine learning community, like CelebA, CIFAR-10 or MNIST. The basis of our discussion and analysis will be [6], where a neural network RG was applied to a holographic setup.

Boundary CFT

Let us first elaborate on the physical theory, we want to model with the network. We will work exclusively with bosonic fields to ensure that the basic implementation is correct and to avoid unnecessary complications. The discretized QFT, which lives on the boundary, is defined on a square lattice. The size can be changed, but for our trainings we used a fixed lattice with 32×32 sites. As in chapter 2, we work in Poincaré coordinates and the boundary coordinates will be denoted by $x, y \in \mathbb{R}^2$ and the radial direction by $z \in \mathbb{R}$. The complex scalar field ψ has the action

$$S_{\text{QFT}}[\psi] = -k \sum_{\langle x, y \rangle} \psi_x^* \psi_y + \sum_x (\mu |\psi_x|^2 + \lambda |\psi_x|^4), \quad (2.8)$$

where the sum $\sum_{\langle x, y \rangle}$ is to be taken over nearest neighbors and $J, \mu, \lambda \in \mathbb{R}$ are global parameters. If one wants to make connection to Witten's description of holography, one can add a further term, which couples the field ψ to the boundary value of a bulk field

$\phi_0 \equiv \phi(z=0)$. Such a source term with minimal coupling has the form

$$S_{\text{src}}[\psi] = -\frac{1}{2} \sum_x (\psi_x^* \phi_{0x} + \phi_{0x}^* \psi_x) . \quad (2.9)$$

In the following we will work with the case of vanishing external source $\phi \rightarrow 0$.

For the parameters we choose $k = 5$, $\mu = -180$, $\lambda = 25$, which creates a mexican hat potential for ψ with $U(1)$ symmetry and a reflection symmetry $\mathbb{Z}_2$, see ???. During training the field $\psi = \rho e^{i\theta}$ will condensate into the minima and is constraint to have a approximate constant amplitude $|\psi| \approx 2 \approx \text{const}$. If the temperature T is sufficiently low, we can ignore the fluctuations perpendicular to the θ direction. Then only one degree of freedom θ survives and the boundary theory becomes an XY-model with action

$$S_{\text{QFT}} \approx -k\rho^2 \sum_{\langle x,y \rangle} \cos(\theta_x - \theta_y) \quad (2.10)$$

at an effective temperature $T = \frac{1}{k\rho^2}$.

Low temperature limit It is important to mention, that this theory is in general not conformally invariant. While the Mermin-Wagner theorem prohibits any spontaneous symmetry breaking in $d \leq 2$, the system can still experience a topological phase transition. That means, it has a critical phase at low temperatures, where it exhibits a quasi-long-range interaction and can thus be described by a CFT. There exist a certain temperature T_{KT} , below which the system can form bound pairs of vortices and goes into an ordered phase. The correlation function will obey a power law instead of an exponential decay, which implies an infinite correlation length. This indicates the existence of conformal symmetry. To check this, we will look for an expression of the two-point correlator in the critical phase. Since for low T the fluctuations of θ will be small, we expand the action (??) up to quadratic order:

$$-J \sum_{\langle x,y \rangle} \cos(\theta_x - \theta_y) = -2NJ + \frac{1}{2}J \sum_{\langle x,y \rangle} (\theta_x - \theta_y)^2 + \dots . \quad (2.11)$$

Within this harmonic approximation we expect small variations of the fields, i.e. $\theta(x) \ll 1$, such that the subtraction can be seen as a derivative by taking the continuum limit. So we arrive at a free field theory for $\theta(x)$ with

$$S_{\text{QFT}} \approx \frac{J}{2} \int d^2x (\nabla\theta(x))^2 . \quad (2.12)$$

The two-point function can be found from the Green's function of the Laplace equation in d dimensions and reads

$$\begin{aligned}\langle \theta(x)\theta(y) \rangle &= -\frac{1}{J} \frac{|x-y|^{2-d}}{(2-d)S_d} + \text{const.} \\ &\xrightarrow{d=2} -\frac{1}{2\pi J} \ln \frac{|x-y|}{a} ,\end{aligned}\tag{2.13}$$

where S_d is the solid angle and a the lattice spacing. The two-point correlation of our original field ψ can then be easily computed:

$$\begin{aligned}\text{Re} \langle \psi_x \psi_y^* \rangle &\sim \text{Re} \langle e^{i(\theta_x - \theta_y)} \rangle \\ &\sim \text{Re} e^{-\frac{1}{2} \langle (\theta_x - \theta_y)^2 \rangle} \\ &\sim \frac{1}{|x-y|^{\frac{1}{2\pi J}}} ,\end{aligned}\tag{2.14}$$

where the second line is valid, since we have a Gaussian distribution due to (??). This power-law is consistent with the result from a CFT. For the XY-model it means, that there exist a ordered phase at low T and the system can undergo a phase transition, the so-called Kosterlitz-Thousless transistion. Our parameter choice leads to a temperature of $T \approx 0.05$, which is much lower than the estimated crititical temperature at $T_{\text{KT}} \approx 0.9$. The emergence of a dual gravity theory in the network can therefore be expected.

Bulk theory

The physical system on the gravity side can potentially contain a large set of different fields. We assume, that on large scales the general behavior can be described effectively by a massive complex scalar field $\phi(x, z)$ with action

$$S_{\text{bulk}}[\phi] = \int d^d x dz \sqrt{g} [g^{\mu\nu} \partial_\mu \phi^* \partial_\nu \phi + m^2 |\phi|^2]\tag{2.15}$$

without any interaction of higher order. Since this action is quadratic in ϕ , we can discretize it by a general ansatz

$$S_{\text{bulk}}[\phi] \approx \sum_{i,j} \phi_i^\dagger \Sigma_{ij}^{-1} \phi_j = \phi^\dagger \Sigma^{-1} \phi ,\tag{2.16}$$

where indices go over the bulk coordinates, i.e. $i = (x, z)$. This leads to a circularly-symmetric complex normal distribution of N random variables

$$p_{\text{bulk}}(\phi) = \frac{1}{\det(\pi\Sigma)} e^{-\phi^\dagger \Sigma^{-1} \phi} . \quad (2.17)$$

The covariance matrix Σ contains the geometric information about the bulk space, so we have to make it learnable by the reparametrization trick. The base distribution will then be chosen to be a complex normal distribution with $\Sigma = 1$, i.e.

$$p_{\text{base}} = \frac{1}{\pi^N} e^{-\varphi^\dagger \varphi} , \quad (2.18)$$

and the bulk field can be obtained by

$$\phi = L\varphi , \quad (2.19)$$

where L can be calculated from the Cholesky decomposition of the covariance matrix $\Sigma = LL^\dagger$. Because this decomposition requires Σ to be positive-definite, we choose to train a lower triangular matrix L directly as this will ensure that Σ will always be positive-definite. While the bulk sample will have the same shape as the boundary data on the lattice, the bulk variables are actually spread in the radial direction z , i.e. they belong to length scales of the RG-flow, see ???. The assignment to their energy scales is dependent on the kernel size of the decimator map.

Nearest neighbor interaction To make the discrete action reflect locality, we can restrict the form of the inverse covariance matrix. by setting all entries to zero if two variables are not nearest neighbors within an RG layer or across different energy scales. Moreover, translation invariance and rotational invariance at constant z can also be implemented if we use the same value for all bulk variables in the same layer. The matrix will then have the form

$$\Sigma^{-1} = \sum_{\langle x, y \rangle} \lambda_{ij} (|i\rangle\langle i| + |j\rangle\langle j| - |i\rangle\langle j| - |j\rangle\langle i|) + mI , \quad (2.20)$$

where I is the unit matrix, m is the mass of the field and $\lambda_{ij} > 0$. This reduces the number of parameters to only two per RG layer. But the covariance matrix will in general not be positive definite without additional constraints on λ_{ij} and m .

Loss function and classical approximation

The probability distribution, that the network produces, can be calculated by a change of variables from the bulk distribution:

$$p_{\text{flow}}(\psi) = p_{\text{bulk}}(\phi) \left| \det \frac{\partial G(\phi)}{\partial \phi} \right|^{-1} \left| \det \frac{\partial G(\phi^\dagger)}{\partial \phi^\dagger} \right|^{-1}. \quad (2.21)$$

Since we are going to take the absolute value of the determinants, the last two factors will give us the same value $|\det J_{G(\phi)}| |\det J_{G(\phi^\dagger)}| = |\det J_{G(\phi)}|^2$. For probability density distillation the reversed KL-divergence

$$D_{\text{KL}}(p_{\text{flow}}(\psi) \parallel p_{\text{QFT}}(\psi)) = \mathbb{E}_\phi [\ln p_{\text{bulk}}(\phi) - 2 \ln |\det J_G| + S_{\text{QFT}}] + \ln Z \quad (2.22)$$

will be used, where the last term is constant and will be dropped in the implementation. The loss function will then be chosen as

$$\mathcal{L} = \mathbb{E}_\phi [\ln p_{\text{bulk}}(\phi) - 2 \ln |\det J_G| + S_{\text{QFT}}[G(\phi)]] \quad (2.23)$$

The QFT will be evaluated on the configuration sampled from the network as $S_{\text{QFT}} = S_{\text{QFT}}[G(\phi)]$.

Let us interpret the above loss function in the context of holography. What does it mean to minimize the KL-divergence? Let's define the distribution for the gravity theory induced by the flow from the CFT density as

$$p_{\text{grav}}(\phi, G) = p_{\text{CFT}}(G(\phi)) |\det J_G|, \quad (2.24)$$

which will depend on the bulk field as well as on the parameters of the flow transformation contained in G . We can think of G as the background geometry, on which the field ϕ lives. As seen in the previous chapter, a training under the reversed KL-divergence is equivalent to training under the forward KL-divergence, where we try to match $p_{\text{grav}}(\phi, G)$ to a prior distribution $p_{\text{bulk}}(\phi)$. The forward divergence is then

$$D_{\text{KL}}(p_{\text{bulk}}(\phi) \parallel p_{\text{grav}}(\phi, G)) = \mathbb{E}_\phi [-S_{\text{bulk}}[\phi] + S_{\text{grav}}[\phi, G]] + \text{const.}, \quad (2.25)$$

where we assumed the form of the probability distribution to be of the form $p \sim e^{-S}$. Since expectation over the bulk distribution gives a constant term, minimizing the loss function is the same as finding the minimum of the gravity action, i.e. the saddle point

approximation. We can make a quick estimation for p_{grav} under training

$$p_{\text{grav}}(G) \equiv \mathbb{E}_{\phi} [p_{\text{grav}}(\phi, G)] \sim \mathbb{E}_{\phi} [e^{-S_{\text{grav}}[\phi, G]}] \geq e^{-\mathbb{E}_{\phi} [S_{\text{grav}}[\phi, G]]} , \quad (2.26)$$

which holds by Jensen's inequality due to the convexity of the exponential. This implies that the marginalized probability of G is maximized under training, i.e. the geometry contained in G is classical. We therefore expect, that fluctuations are of small order.

Disentangled and decimators

For the disentanglers and decimators filter D , one can in principle choose any map, from the point of view of machine learning. We can implement even entire networks like RNVPs, as long as their Jacobian and inverse transformation can be easily calculated. In our case, D is implemented by a single layer, which takes n variables. We can say that the kernel size of the filter is $\sqrt{n} \times \sqrt{n}$. The logarithm of the Jacobian determinants (LDJ) will be collected along with each transformation. We then only have to sum up all LDJs to calculate the loss function. A general filter will contain a scaling, a linear map and an activation function:

$$D = a \circ M \circ s \quad (2.27)$$

The network's total transformation G will consists of multiple layers $G_i(D_{\text{decim}}^i, D_{\text{disent}}^i)$ at each RG scale.

Scaling In the direction of $G(\phi)$ the n incoming variables ϕ_i for one filter will be scaled first by

$$s(\phi_i) = e^{\lambda_i} \phi_i \quad (2.28)$$

with inverse

$$s^{-1}(\psi) = e^{-\lambda_i} \psi_i \quad (2.29)$$

The LDJ will simply be given by the exponent λ_i .

Linear map As the most general case we implement a linear layer of size (n, n) from the PyTorch library. The inverse and the LDJ can be easily calculated using the linear algebra methods of PyTorch. The unitary and orthogonal matrices do not contribute to the loss function, because their LDJ is simply 0 by construction. We choose these types of matrices for M :

- Linear(n,n)
- SU(n) from Cayley transform of a skew-hermitian matrix with zero trace
- SU(n) from matrix exponential of a skew-hermitian matrix with zero trace
- stacked O(2) matrices (only for $n = 4$)

It is important to note, that the first three maps have complex weights in general and the last one has real entries.

The stacked O(2) matrices are used in [6]. They can be build from two 4×4 block-diagonal matrices, which contain $O(2)$ transformations,

$$A(\vartheta_i, \vartheta_j) = \begin{pmatrix} \cos \vartheta_i & -\sin \vartheta_i & 0 & 0 \\ \sin \vartheta_i & \cos \vartheta_i & 0 & 0 \\ 0 & 0 & \cos \vartheta_j & -\sin \vartheta_j \\ 0 & 0 & \sin \vartheta_j & \cos \vartheta_j \end{pmatrix} \quad (2.30)$$

$$B(\vartheta_i, \vartheta_j) = \begin{pmatrix} \sin \vartheta_i & \cos \vartheta_i & 0 & 0 \\ \cos \vartheta_i & -\sin \vartheta_i & 0 & 0 \\ 0 & 0 & \sin \vartheta_j & \cos \vartheta_j \\ 0 & 0 & \cos \vartheta_j & -\sin \vartheta_j \end{pmatrix} \quad (2.31)$$

along with a matrix to swap two entries of the input vector

$$C = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (2.32)$$

The disentangler and decimator matrices are then defined as

$$M_{\text{disent}}^{O(2)} = A(\vartheta_6, \vartheta_7) \circ C \circ A(\vartheta_4, \vartheta_5) \circ A(\vartheta_2, \vartheta_3) \circ C \circ A(\vartheta_0, \vartheta_1) \quad (2.33)$$

$$M_{\text{decim}}^{O(2)} = B(\vartheta_6, \vartheta_7) \circ C \circ B(\vartheta_4, \vartheta_5) \circ A(\vartheta_2, \vartheta_3) \circ C \circ A(\vartheta_0, \vartheta_1). \quad (2.34)$$

Non-linearity As an activation function we choose the one, that is used in [6]. It is only properly defined for $|\phi_i| \neq 0$.

$$a(\phi_i) = \frac{\phi_i}{|\phi_i|} \operatorname{arsinh}|\phi_i| \quad (2.35)$$

with an LDJ

$$\log \left| \det \frac{\partial a}{\partial \phi_i} \right| = -\frac{1}{2} \ln(1 + |\phi_i|^2) . \quad (2.36)$$

In the renormalizing direction the activation is given by

$$a^{-1}(\psi_i) = \frac{\psi_i}{|\psi_i|} \sinh|\psi_i| \quad (2.37)$$

with

$$\log \left| \det \frac{\partial a^{-1}}{\partial \psi_i} \right| = \ln(\cosh |\psi|) \quad (2.38)$$

The activation contains no parameter for training. The sinh was chosen, since it naturally appears in hyperbolic spaces.

2.3 Training

As mentioned before, the flow network will be trained in two stages. In the first, the Gaussian bulk will be fixed with covariance matrix $\Sigma = 1$. This means, we will try to minimize bulk correlation as much as possible in stage I training. As some correlation will be still left after training, we interpret this as a part of dynamics of the bulk system, i.e. it should be part of *Sigma*. We then will fix the learned network and train only the covariance matrix of the correlated Gaussian. This then should encode the geometry of the network into the bulk action.

We first train several combinations of filters D for disentanglers and decimators with different linear maps M . Our standard hyper-parameters are:

CFT size	32×32
kernel size of D	2×2
batch size	1024
steps stage I	100k
steps stage II	100k

The learning rate for the scaling is set to be 10^{-2} in the beginning and then decreased by

one order of magnitude every $\frac{1}{3}$ of the training steps, e.g. every 20000 steps in our setup. The learning rate for the filters D are fixed at 10^{-4} . The training of Σ will be done with learning rate 10^{-3} .

Weight initialization

The weights of the scaling layers, are set to 0 at the start of stage I training. This way we can prevent an exploding loss function.

Whenever possible, we initialize the weights as in [6], because that reproduces the EHM, which was used in [11]. This seems to be a good choice, as we know that this setup could produce a sensible RG-scheme. The initialized weights are

$$M_{\text{disent}}^{\text{init}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (2.39)$$

and

$$M_{\text{decim}}^{\text{init}} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & -1 \end{pmatrix}. \quad (2.40)$$

As the $SU(n)$ layers are build as parametrized linear layers by using the parametrization class of PyTorch, we have to map the initialized weights back to the weights of the original linear layer by the reverse parametrization. For disentanglers with Cayley map this is not possible, since $M_{\text{disent}}^{\text{init}}$ is not a solution of the inverse Cayley transform. The $SU(n)$ matrix from the exponential map cannot be initialized at all in general, because there is no inverse map, i.e. the matrix logarithm is not implemented in PyTorch. In these cases we will simply use the default random initial weights. All layers will have no bias term, so they are compatible with the $U(1)$ symmtery of the boundary theory.

For training stage II, the covariance matrix Σ or Cholesky matrix L should not be initiated as a identity matrix. Otherwise only the diagonal components will be trained. For our setup we choose to train mainly with the approach of directly using the Cholesky matrix L , because the definition of Σ^{-1} with nearest-neighbour interactions will not stay positive-definite during training. The matrix L will be then initialized after training stage I with

a small perturbation on the off-diagonals as

$$L^{\text{init}} = \begin{pmatrix} 1 & & 0 \\ & \ddots & \\ \epsilon & & 1 \end{pmatrix}. \quad (2.41)$$

For the nearest-neighbor approach, the same has to be done. The off-diagonal components should not be initialized at zero, but as small values. Here, one should be careful when choosing the values for λ_{ij} , such that the covariance matrix is positive-definite.

Training times We are training on a Nvidia GeForce GTX 1080. CPUs are from the AMD EPYC 7000 series on a cluster. We can see, that the unitary parametrization

disentangler	decimator	CPU	GPU
Linear	Linear	50k	13k
Linear	SU(4) Cayley	50k	13k
SU(4) Cayley	SU(4) Cayley	50k	13k
Linear	SU(4) Exp	50k	13k
Linear	stacked O(2)	190k	19k

Table 2.1: Approximate training times in seconds

of the linear layer have no significant impact on the performance. Only the stacked O(2) transformation is much slower, because of its custom design. This however heavily depends on the specific implementation.

The loss function will have the behavior seen in ???. After the initial drop we see, that it is only slowly decreasing as usual in neural network training. Another drop in the loss can be achieved, when we enter stage II training. The covariance matrix of the bulk distribution seems to be capturing some correlation features, which could not be resolved by training the network weight alone.

2.4 Results

Variation of the filter

To check whether the training was successful, we analyze the configuration of boundary samples. We look at the distribution of the field values ψ_x in the complex plane and on the lattice in real space. The configuration on the lattice is expected to show vortices and

some kind of ordering resembling symmetry breaking. The $U(1)$ symmetry should be restored look at sufficiently large batch of samples. Moreover, we calculate the two-point function for various distances to see, if it matches our power-law behavior from CFT. We plot the real, imaginary part and absolute value of the correlation. In most cases the imaginary part doesn't have any smooth behavior over distance.

While the training can result in expected configurations for the field theory, we frequently observe mode collapse. The systems seems to prefer a configuration that is either anti-ferromagnetic or has a periodic structure, see ???. Once the model has reached such a state, it is not able to produce any other configuration as training goes on. This happens especially often, when choosing the general linear layer for both filters. The two-point function in this case can be fluctuating, see ???.

A more stable setup is to choose for the decimator a unitary map, either by using the Cayley map or the matrix exponential. We find in particular, that using the Cayley map gives us the most reliable results, see ???. The correlation will then decay smoothly with distance.

For the case that we use a unitary map for disentangler and decimators, we get the quickest convergence. However the training again seems to be unstable, as we produce more periodic configurations.

The stacked $O(2)$ transformation also experiences mode collapse. Moreover it is not easy to generalize it to other kernel sizes.

Bias term

The bias term should be set to 0, because otherwise it would break the rotation symmetry. This leads to a mode collapse, as the network will pick a preferred direction during training. The $U(1)$ is not restored over multiple samples. If we want to include an external magnetic field ϕ_0 , a non-zero bias term is necessary to capture this effect, see ???.

Different kernel size

For this test we use the most reliable choice of We observe no noticeable differences in the training times. This is due to the fact, that although the parameter count per layer is increased, we also have less RG-layers. When using bigger kernel sized we could not

disentangler	Linear(4,4)
decimator	SU(4) Cayley

produce a sensible boundary configuration. In fact we get a peridiodic configuration and the range of correlation scales with kernel size.

2.5 Holographic distances

The distance in the bulk space can be inferred from the two-point correlator on large scales. If the bulk field is massive, the correlator will decay exponentially with the distance between two points in the bulk

$$\mathcal{C}(\phi_i, \phi_j) \sim \exp \left[-\frac{\mathcal{D}_{\text{holo}}(\phi_i, \phi_j)}{\xi} \right] , \quad (2.42)$$

where the correlation is defined as

$$\mathcal{C}(\phi_i, \phi_j^*) = \langle \phi_i \phi_j^* \rangle - \langle \phi_i \rangle \langle \phi_j^* \rangle . \quad (2.43)$$

We use this as the definition of our geodesic distances as in [11].

$$\mathcal{D}_{\text{geo}}(\phi_i, \phi_j) = -\xi \ln \frac{|\langle \phi_i \phi_j^* \rangle|}{C_0} \quad (2.44)$$

This definition depends on the operator or field, that we use. By using the mutual information however, we can put an upper bound on it [14].

$$I(\phi_x : \phi_y) \geq \frac{C(\phi_x, \phi_y)^2}{2\|\phi_x\|^2\|\phi_y\|^2} \quad (2.45)$$

The mutual information is defined to be

$$I(\phi_x : \phi_y) \equiv H(\phi_x) + H(\phi_y) - H(\phi_x, \phi_y) \quad (2.46)$$

Let us calculate the mutual information for a bivariate complex Gaussian, to get a formula for the information between two variables. This will be the effective distribution, which contains the correlation between two bulk sites. It can be defined as a marginalized distribution

$$p(\phi_i, \phi_j) = \sum_{k \neq i, j} p(\{\phi_k\}) . \quad (2.47)$$

The joint entropy for a multivariate normal distribution of N variables between ϕ_i and ϕ_j

$$\begin{aligned}
H(\phi) &= - \int d\phi d\phi^* p \ln p \\
&= \ln |\det(\pi\Sigma)| + \langle \phi^\dagger \Sigma^{-1} \phi \rangle \\
&= \ln |\det(\pi\Sigma)| + \langle \text{tr}(\phi \phi^\dagger \Sigma^{-1}) \rangle \\
&= N \ln(\pi) + \ln |\det(\Sigma)| + N
\end{aligned} \tag{2.48}$$

since $\langle \phi \phi^\dagger \rangle = \Sigma$. Plugging it into equation (??) gives us

$$I(\phi_i : \phi_j) = \ln \left(1 - \frac{\langle \phi_x^* \phi_y \rangle \langle \phi_y^* \phi_x \rangle}{\langle \phi_x^* \phi_x \rangle \langle \phi_y^* \phi_y \rangle} \right) \tag{2.49}$$

So we can get another formula for the geodesic distance:

$$\mathcal{D}(\phi_x, \phi_y) = -\xi \ln \frac{I(\phi_x : \phi_y)}{I_0} . \tag{2.50}$$

The correlation $\langle \phi_i \phi_j^* \rangle$ can be found as the entry in the covariance matrix $\langle \phi_i \phi_j^* \rangle = \Sigma_{ij}$ or by direct computation from a batch of sampled bulk states $\langle \phi_i \phi_j^* \rangle = \mathbb{E}_\phi[\phi_i \phi_j^*]$.

In our analysis we choose to use the definition (??), since here the distance can be made sensible even between the same variable. The offset C_0 is chosen as the average value of the diagonal entries of the covariance. We observe, that in general the diagonal entries will be much large than the off-diagonals and are of very similiar value for every variable, so

$$C_0 = \frac{1}{N} \sum_i |\langle \phi_i \phi_i^* \rangle| \approx |\langle \phi_i \phi_i^* \rangle| \tag{2.51}$$

This choice of the constant will set the holographic distance to 0, if we compute

$$\mathcal{D}_{\text{geo}}(\phi_i, \phi_i) = -\xi \ln \frac{|\langle \phi_i \phi_i^* \rangle|}{C_0} \approx 0 \tag{2.52}$$

for any ϕ_i .

As we see from figure ??, the latent bulk space will not be evenly covered by the bulk variables. In order to define a holographic distance in a consistent way, we define the holographic distance to be evaluated between different patches covered by a decimator filter. The distance between two filter will then be the average geodesic distance between

the variables associated to it.

$$\mathcal{D}_{\text{holo}}(A, B) = \text{avg}_{\phi_i \in A, \phi_j \in B} \mathcal{D}_{\text{geo}}(\phi_i, \phi_j) \quad (2.53)$$

To probe the geometry we will calculate the angular distance within one layer and the radial distance across different z . The radial holographic distance has to be proportional to the number of layers separating two decimators. For the angular distance we expect for a hyperbolic space a logarithmic dependence on the difference in the coordinates as

$$\mathcal{D}_{\text{holo}}(x, y) \sim \ln |x - y| , \quad (2.54)$$

where x, y are the position of two decimators within a layer.

In our implementation we however fail to reproduce this hyperbolic structure, see ?? and ?. Moreover, when calculating the holographic distance via the bulk samples instead of the covariance matrix, we observe high fluctuations in the distance function. Apparently the network has not learned a stable configuration for the bulk field. Or in other words geometry seems to not have reached its classical limit. One potential problem is the mode collapse, that frequently occurs during training. This should correspond to a local minimum in the loss function. So although the network appears to be trained, the saddle point of the induced gravity action S_{grav} has not been reached.

Bibliography

- [1] N. Bao et al. “Consistency Conditions for an AdS/MERA Correspondence”. In: *Phys. Rev. D* 91, 125036 (2015) (2015). DOI: 10.1103/PhysRevD.91.125036. arXiv: 1504.06632 [hep-th].
- [2] G. Evenbly and G. Vidal. “Tensor network states and geometry”. In: *J Stat Phys* (2011) 145:891-918 (2011). DOI: 10.1007/s10955-011-0237-4. arXiv: 1106.1082 [quant-ph].
- [3] K. Hashimoto. “AdS/CFT as a deep Boltzmann machine”. In: *Phys. Rev. D* 99, 106017 (2019) (2019). DOI: 10.1103/PhysRevD.99.106017. arXiv: 1903.04951 [hep-th].
- [4] K. Hashimoto et al. “Deep Learning and AdS/CFT”. In: *Phys. Rev. D* 98, 046019 (2018) (2018). DOI: 10.1103/PhysRevD.98.046019. arXiv: 1802.08313 [hep-th].
- [5] K. Hashimoto et al. “Deep Learning and Holographic QCD”. In: *Phys. Rev. D* 98, 106014 (2018); *GitHub*: https://github.com/AkinoriTanaka-phys/DL_holographicQCD (2018). DOI: 10.1103/PhysRevD.98.106014. arXiv: 1809.10536 [hep-th].
- [6] H.-Y. Hu et al. “Machine Learning Holographic Mapping by Neural Network Renormalization Group”. In: *Phys. Rev. Research* 2, 023369 (2020) (2019). DOI: 10.1103/PhysRevResearch.2.023369. arXiv: 1903.00804 [cond-mat.dis-nn].
- [7] H.-Y. Hu et al. “RG-Flow: A hierarchical and explainable flow model based on renormalization group and sparse prior”. In: *Mach. Learn.: Sci. Technol.* 3 035009 (2022) (2020). DOI: 10.1088/2632-2153/ac8393. arXiv: 2010.00029 [cs.LG]. URL: <https://github.com/hongyehu/RG-Flow>.
- [8] C. H. Lee and X.-L. Qi. “Exact holographic mapping in free fermion systems”. In: *Phys. Rev. B* 93, 035112 (2016) (2015). DOI: 10.1103/PhysRevB.93.035112. arXiv: 1503.08592 [hep-th].
- [9] S.-H. Li and L. Wang. “Neural Network Renormalization Group”. In: *Phys. Rev. Lett.* 121, 260601 (2018) (2018). DOI: 10.1103/PhysRevLett.121.260601. arXiv: 1802.02840 [cond-mat.stat-mech]. URL: <https://github.com/li012589/NeuralRG>.

- [10] G. Papamakarios et al. “Normalizing Flows for Probabilistic Modeling and Inference”. In: *Journal of Machine Learning Research*, 22(57):1-64, 2021 (2019). arXiv: 1912.02762 [stat.ML].
- [11] X.-L. Qi. “Exact holographic mapping and emergent space-time geometry”. In: (2013). arXiv: 1309.6282 [hep-th].
- [12] D. J. Rezende and S. Mohamed. “Variational Inference with Normalizing Flows”. In: *Proceedings of Machine Learning Research* 37 (2015). arXiv: 1505.05770 [stat.ML].
- [13] G. Vidal. “A class of quantum many-body states that can be efficiently simulated”. In: *Phys. Rev. Lett.* 101, 110501 (2008) (2006). DOI: 10.1103/PhysRevLett.101.110501. arXiv: quant-ph/0610099 [quant-ph].
- [14] M. M. Wolf et al. “Area laws in quantum systems: mutual information and correlations”. In: *Phys. Rev. Lett.* 100, 070502 (2008) (2007). DOI: 10.1103/PhysRevLett.100.070502. arXiv: 0704.3906 [quant-ph].